



ROI Training

Anthropic Models on Amazon Bedrock

Module 3:

Data Preparation and
Knowledge Bases for RAG



Lab 1

Course Contents

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

► **Module 3: Data Preparation and Knowledge Bases for RAG**

Module 4: Tool Use and Agentic Workflows with Claude

Module 5: Security, Governance, and Guardrails

Module 6: Practical Architecture and Deployment Patterns

Module 7: Claude Code: Building Bedrock Applications

Module 8: Amazon Kiro and Enterprise Code Generation



Module Objectives



RAG Architecture

Understand RAG benefits for enterprise applications



Knowledge Bases

Set up Bedrock Knowledge Bases with data sources



Retrieval Queries

Query knowledge bases with citation support



Optimize RAG Performance

Apply reranking and chunking strategies



ROI Training

RAG Architecture and Concepts

Knowledge Bases for Amazon Bedrock



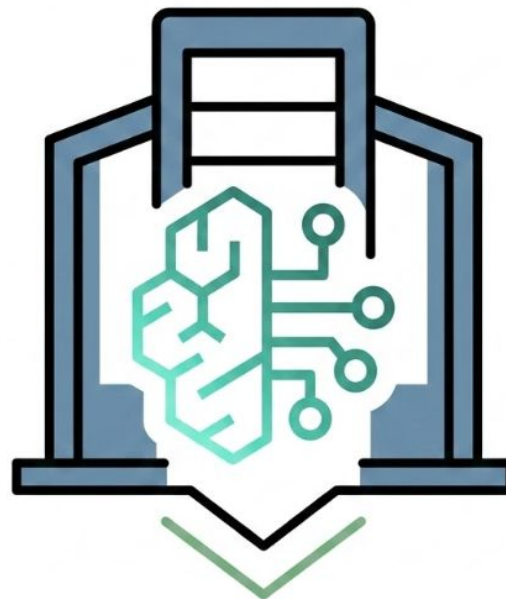
Managed RAG for
Amazon Bedrock

Connect your data to foundation models without code
Automatic document chunking, embedding, and retrieval
Fully managed vector store with multiple engines
Query via API or use directly in your application

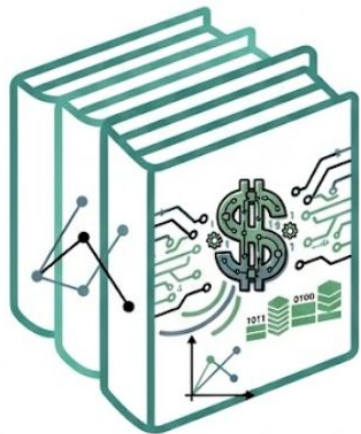
The Enterprise AI Challenge

The Gap

- Claude doesn't know your internal policies
- Proprietary docs aren't in training data
- Fine-tuning is expensive, needs retraining
- Context windows have token/cost limits
- Solution: Give Claude what it needs, when needed



What is RAG?



Retrieval-Augmented
Generation

Retrieval: Find relevant info from your data

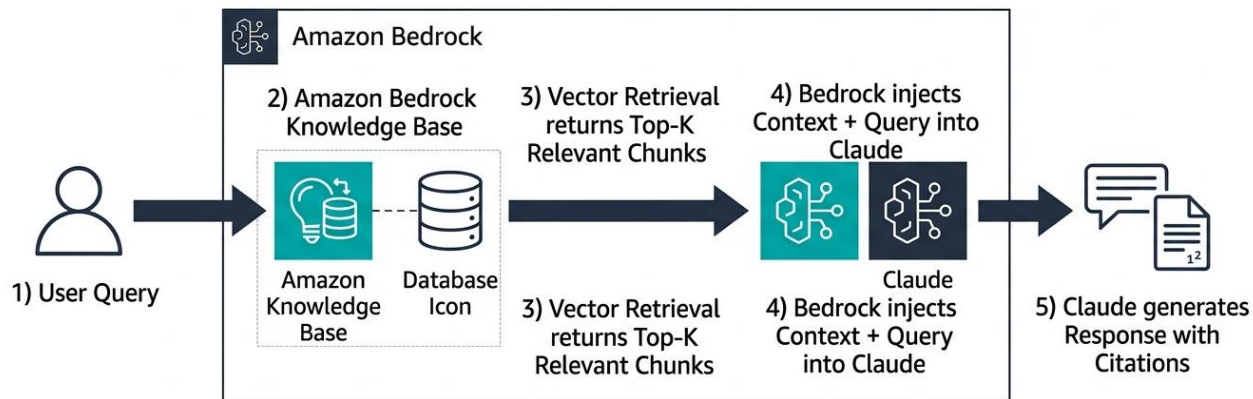
Augmented: Add that info to prompt context

Generation: Claude responds using retrieved data

Strategic context use — like a librarian, not a memorized encyclopedia

RAG Architecture Overview

AWS Architecture Diagram: RAG Flow on Amazon Bedrock



Why RAG for Enterprise?

Accuracy

Answers grounded in
your documents

Currency

Updates without
retraining the model

Control

You decide what
the model accesses

Cost

Only retrieve
what's needed

Vector Embeddings Explained

Embeddings 101

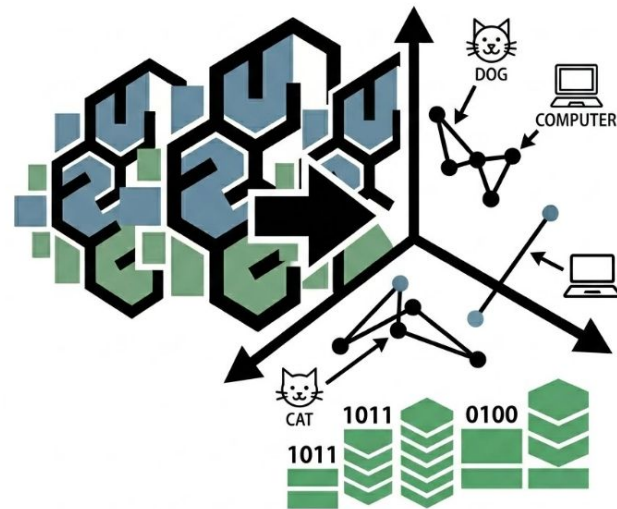
Text converted to numerical vectors

Similar meanings produce similar vectors

'Return policy' close to 'refund process'

Retrieval = finding vectors closest to query

Semantic search: by meaning, not keywords



Knowledge Bases for Amazon Bedrock

1

Data Sources: S3, web crawlers, structured databases

2

Embeddings: Auto-conversion via Titan or other models

3

Vector Store: OpenSearch Serverless or bring your own

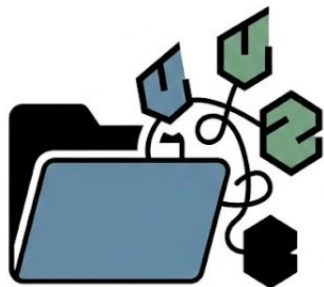
4

Retrieval API: Query interface with citation support

5

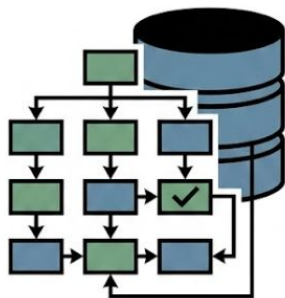
Fully Managed: No infrastructure to provision

Supported Data Types



Unstructured

PDFs, Word docs
Text, HTML, and
Markdown files



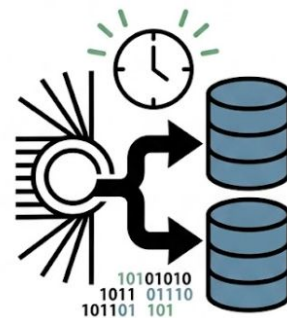
Structured

Database
connections
with natural language
to SQL queries



Multimodal

Docs with images
plus image-based
content search



Real-time

Immediate ingestion
for live data sources
as they update



ROI Training

Configuring Amazon Bedrock Knowledge Bases

Knowledge Base Configuration Steps

1

Name and description — Be descriptive for multiple KBs

2

IAM role — Permissions to read from data sources

3

Data source — S3 bucket, prefix, sync schedule

4

Embedding model — Amazon Titan, Cohere, or custom

5

Vector store — OpenSearch Serverless or your own

Chunking Strategies

Default

Bedrock decides how to split documents for most workloads
No configuration needed

Fixed-Size

You specify the chunk size in tokens for precise control over retrieval granularity

Semantic

Chunks split at natural content boundaries like paragraphs and sections for best meaning fit

Embedding Model Options

Model	Best For	Notes
Amazon Titan Embeddings V2	Most use cases	Default, works well for English
Titan Multimodal Embeddings	Documents with images	Search across text and visuals
Cohere Embed	Multilingual content	Strong non-English support
Custom models	Special requirements	Bring your own if needed

Vector Store Options



OpenSearch

Serverless and fully managed by AWS
Recommended default



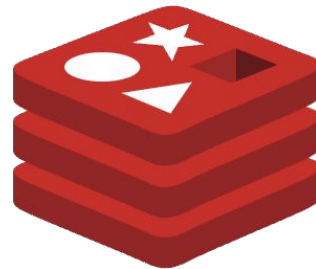
Aurora pgvector

PostgreSQL-based vector storage with relational features



Pinecone

Hosted vector DB for cross-app sharing and existing setups



Redis

In-memory vector store for low-latency retrieval workloads

Data Sync Options

1

Manual sync: Trigger on-demand from console or API

2

Automatic sync: Schedule regular updates (hourly, daily)

3

Real-time ingestion: Documents available immediately

4

Incremental updates: Only process changed files

5

Plan for sync time — not instantaneous in app design

Testing in the Console

The screenshot displays the Amazon Bedrock console interface. The browser address bar shows the URL: `https://us-east-1.console.aws.amazon.com/bedrock/home/browse/brotk/knowledge-base-1-query=1j63663326-16n%adered_to...`. The console header includes the AWS logo, a search bar, and navigation links for Bedrock and Knowledge bases. The breadcrumb trail indicates the current path: `Bedrock > Knowledge bases > my-kb > Test`.

The main content area is divided into two panels. The left panel, titled "Test knowledge base", contains a "Prompt" section with a text input field containing the question "What is our refund policy?". Below the input field is an orange "Test" button. The "Model" section shows a dropdown menu with "Claude v2" selected and "Anthropic" listed below it. Underneath, there are three expandable sections: "Retrieval parameters", "Retrieval parameters", and "Advanced settings".

The right panel, titled "Response", displays the model's output. It begins with a paragraph: "Based on the provided sources, our standard refund policy allows customers to return products within 30 days of purchase for a full refund, provided the items are in original condition. Refunds are typically processed back to the original payment method within 5-7 business days. For personalized or sale items, different policies may apply." Below this, the "Citations" section lists four references:

- [1] [Refund Policy Overview](#) (refund_policy.pdf, page 2)
- [2] [General Terms and Conditions](#) (terms_and_conditions.docx, section 5.1)
- [3] [General Terms and Conditions](#) (terms_and_conditions.docx, section 5.1)
- [4] [General Terms and Conditions](#)

The footer of the console shows the "Bedrock" logo, a "Demand" dropdown, and copyright information: "© 2022 Amazon 1-Console.aws.amazon.com, Inc." along with links for "Privacy" and "Terms & account".

Demo



- Create a knowledge base in the Bedrock console
- Configure S3 data source and sync documents
- Test RAG queries with natural language questions
- Review retrieved chunks, scores, and citations



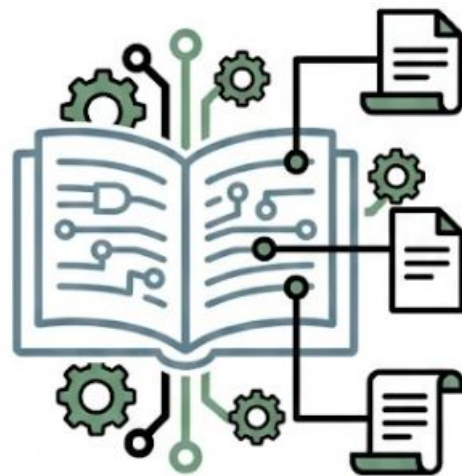
ROI Training

Optimization and Best Practices

Citation Support for Source Verification

Source Attribution

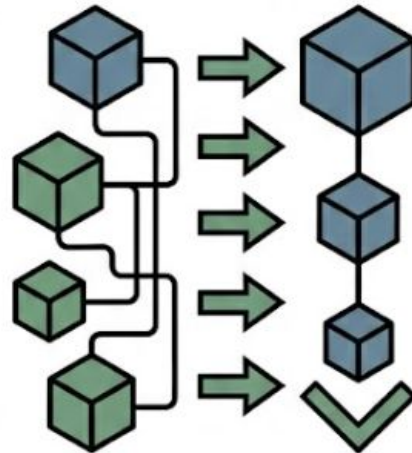
- Automatic source attribution per response
- Document name, page, chunk location
- Users verify by clicking through citations
- Critical for compliance and regulated industries
- Enabled by default in RetrieveAndGenerate API



Reranking for Better Results

Better Retrieval

- Initial retrieval: fast vector similarity
- Reranking: second-pass relevance scoring
- Models: Amazon Rerank, Cohere Rerank
- Trade-off: 100-300ms additional latency
- Worth it for most applications



Chunking Best Practices

1

Too small: Context is fragmented, meaning is lost

2

Too large: Retrieval is imprecise, context window fills up

3

Sweet spot: 200-500 tokens with overlap

4

Overlap: Ensures important context isn't split

5

Semantic chunking: Break at natural boundaries when possible

Query Optimization Techniques

1

Reformulation: Rewrite vague queries first

2

Metadata filtering: Narrow by source or date

3

Hybrid search: Vectors plus keyword matching

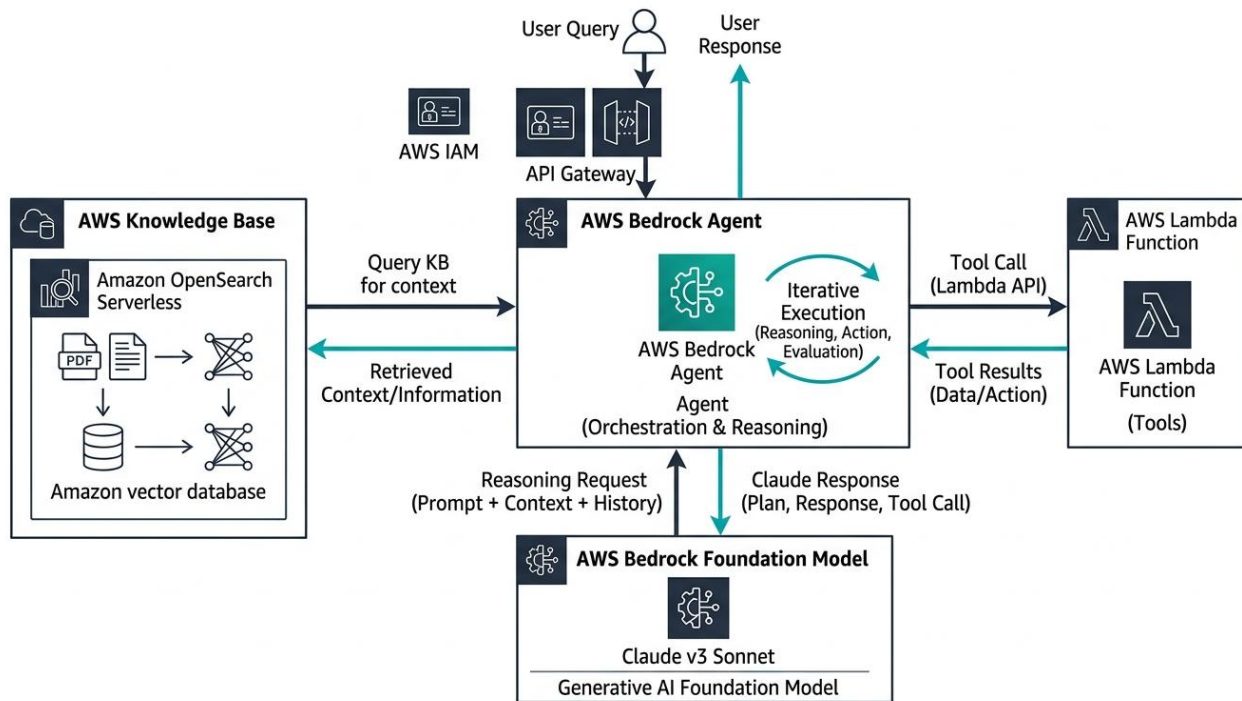
4

Result count: Balance relevance vs. completeness

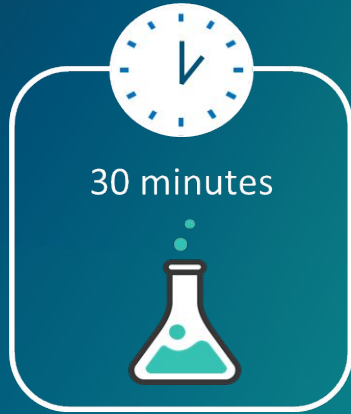
5

Iterate: Test with real user questions early

Knowledge Base + Agent Integration



Lab 1



Claude on Amazon Bedrock with RAG

In this lab, you will:

- Differentiate and invoke Claude Models on Amazon Bedrock
- Observe latency and token usage
- Create a knowledge base from sample documents
- Verify accuracy of RAG citations

What We Covered



RAG Architecture

Understand RAG benefits for enterprise applications



Knowledge Bases

Set up Bedrock Knowledge Bases with data sources



Retrieval Queries

Query knowledge bases with citation support



Optimize RAG Performance

Apply reranking and chunking strategies

Module 4 Is Next!

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

Module 3: Data Preparation and Knowledge Bases for RAG

► **Module 4: Tool Use and Agentic Workflows with Claude**

Module 5: Security, Governance, and Guardrails

Module 6: Practical Architecture and Deployment Patterns

Module 7: Claude Code: Building Bedrock Applications

Module 8: Amazon Kiro and Enterprise Code Generation

